Task 4: Event-Handling Forensics – Drawing Pad

Task 4.1 – Defect

The method is written mouseclicked, but MouseAdapter defines mouseClicked with a capital C. The method therefore does not override the callback and is never invoked. The label is also not updated in the original code.

Task 4.2 – Failing interaction

Click several times, minimise the window and restore it. Dots disappear because getGraphics() draws directly to the screen surface; repainting clears that surface and the application has not retained the points.

Task 4.3 – Repair

Store each click in a List<Point>, update the label, call repaint(), and override paint(Graphics) to redraw every stored point. The source below completes TODO 1 and TODO 2.

Task 4.4 – Prevent recurrence

@Override asks the compiler to verify that the method really overrides a superclass method. Misspelling mouseClicked causes a compile-time error.

Alternative and complexity

A BufferedImage can retain pixels and paint() can copy the image to the screen. A point list needs O(n) work per repaint and O(n) point storage. An image needs O(width×height) pixel storage and one image copy per repaint; drawing each new dot changes only a small region.

Java Source Code

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.List;
public class DrawPad extends Frame {
    private final List<Point> points=new ArrayList<>();
    private final Label status=new Label("Clicks: 0");
    public DrawPad(){
        super("Draw Pad");add(status,BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter(){
            @Override public void mouseClicked(MouseEvent e){
                points.add(e.getPoint());status.setText("Clicks: "+points.size());repaint();
            }
        });
        addWindowListener(new WindowAdapter(){@Override public void windowClosing(WindowEvent e){dispose();}});
        setSize(400,300);
    }
    @Override public void paint(Graphics g){
        super.paint(g);
        for(Point p:points)g.fillOval(p.x-3,p.y-3,6,6);
    }
    public static void main(String[] a){new DrawPad().setVisible(true);}
}
```

Input Files

No external input file is required; enter values in the GUI.

Output Screenshot

Illustrative console-style output in the requested black background, white text and green status theme. For GUI tasks, add genuine screenshots of the actual window and interactions.

```
Task 4 – Illustrative Console

Draw Pad opened
Click 1 recorded
Click 2 recorded
Click 3 recorded
Clicks: 3
Window minimised and restored
All 3 dots remain visible

Press ENTER to exit console.
```

Compile and Run

Open terminal in src, then run:
javac DrawPad.java
java DrawPad

README Result Note

Record your Java version, actual execution result, test observations, screenshots and GitHub commit in README.md.